

Progress Report

Day 1 — Project Setup & Architecture Planning

Task Management MERN Stack Platform

Shafin Nigamana | 19 May 2026

1. Project Overview

This Project focuses on building a team task management platform using the MERN stack. The application will allow teams to create, assign, and track tasks, with user authentication and activity logging built in.

Project name: Task Management MERN

Repository: github.com/ShafinNigamana/task-management-mern (Private)

Stack: MongoDB Atlas, Express.js, React (Vite), Node.js

Repo structure: Monorepo — /client, /server, /docs

Branching: main + feature branches (Git Flow)

2. Day 1 Objective

The goal for Day 1 was to get the project foundation in place before any actual feature development starts. This included setting up the repository, defining the folder structure, documenting the system architecture, and establishing coding standards that the project will follow going forward.

3. Tasks Completed

Repository & Structure

- Initialized GitHub repository (task-management-mern)
- Created monorepo folder structure: /client, /server, /docs
- Added .gitignore and README.md with project description and tech stack
- Set up Git branching — main branch as base, feature/project-setup as working branch
- Pushed initial commits to remote, verified SSH authentication with GitHub

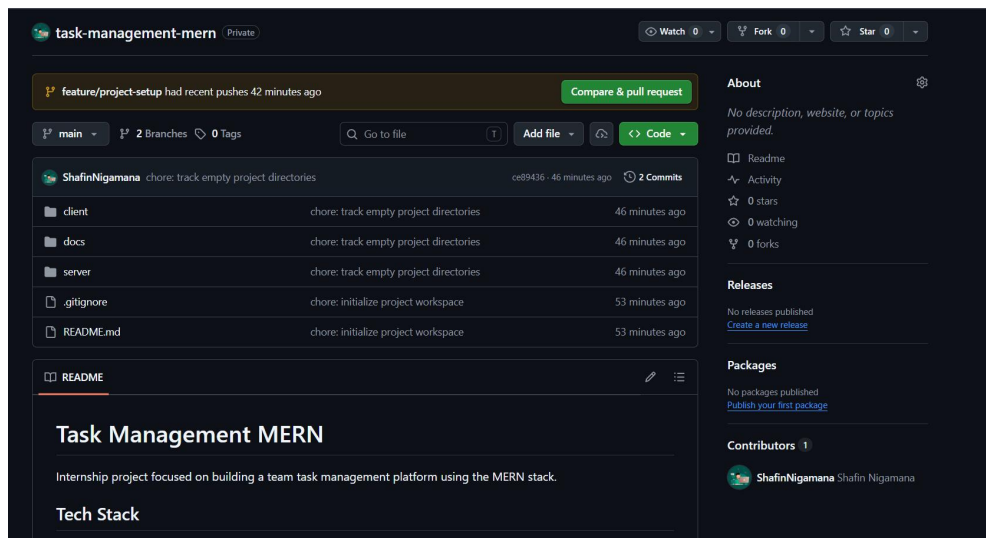


Figure 3.1 — Initial GitHub repository setup and project structure.

Architecture & Documentation

- Created system architecture diagram (docs/architecture/architecture-diagram_d1.png)
- Wrote docs/architecture/system-overview.md — covers component roles and data flow
- Wrote docs/coding-standards.md — defines conventions to follow across the codebase

4. Architecture Decisions

The following decisions were made on Day 1 and documented in the system overview.

Frontend

- React with Vite — chosen for fast dev builds and straightforward project setup
- Will communicate with the backend via REST API using JWT-based authentication

Backend

- Express.js on Node.js — handles API routing and business logic
- JWT used for auth token generation and verification

Data Stores

- MongoDB Atlas — primary database for Users, Teams, and Tasks (operational data)
- MySQL Database — used separately for Audit Activity Logs

MongoDB was selected for flexible operational data, while MySQL was separated for structured audit logging.

5. System Architecture Diagram

Team Task Management Platform

System Architecture

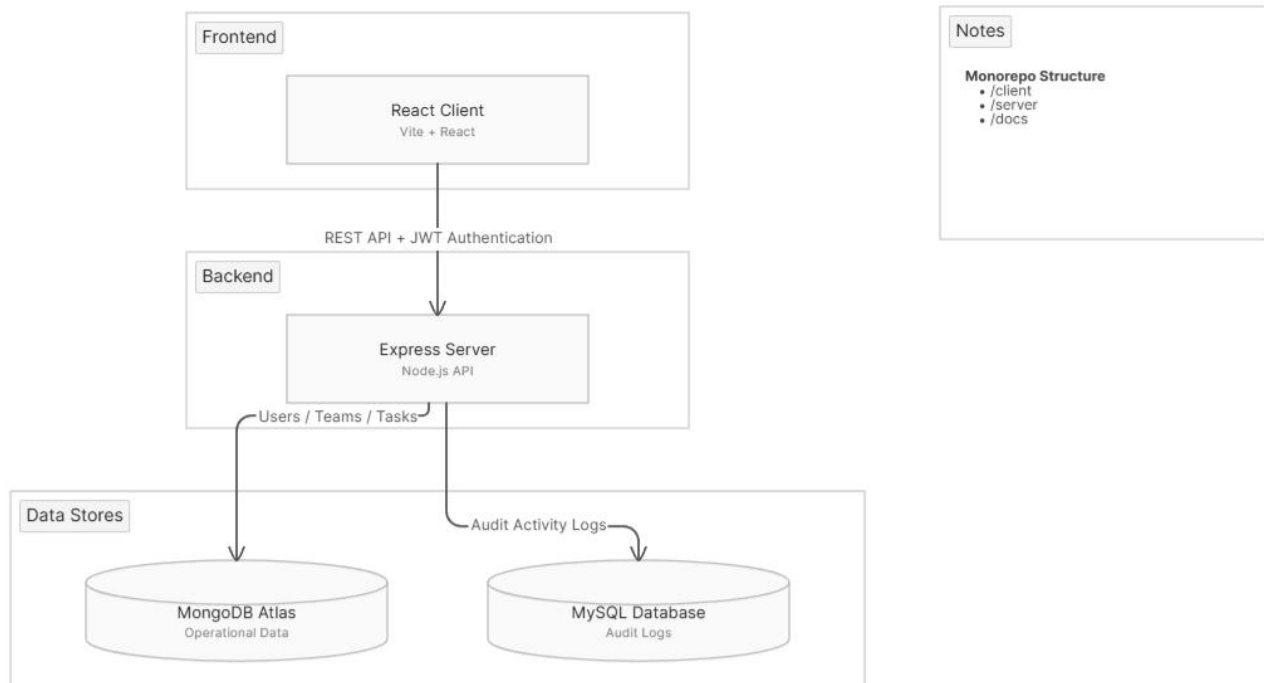


Figure 5.1 — Initial system architecture for the Task Management MERN platform.

6. Repository Structure

Current state of the repository after Day 1:

task-management-mern/

```
├── client/
│   └── .gitkeep
├── docs/
│   ├── architecture/
│   │   ├── architecture-diagram_d1.png
│   │   └── system-overview.md
│   ├── daily-logs/
│   └── coding-standards.md
├── server/
│   └── .gitkeep
├── .gitignore
└── README.md
```

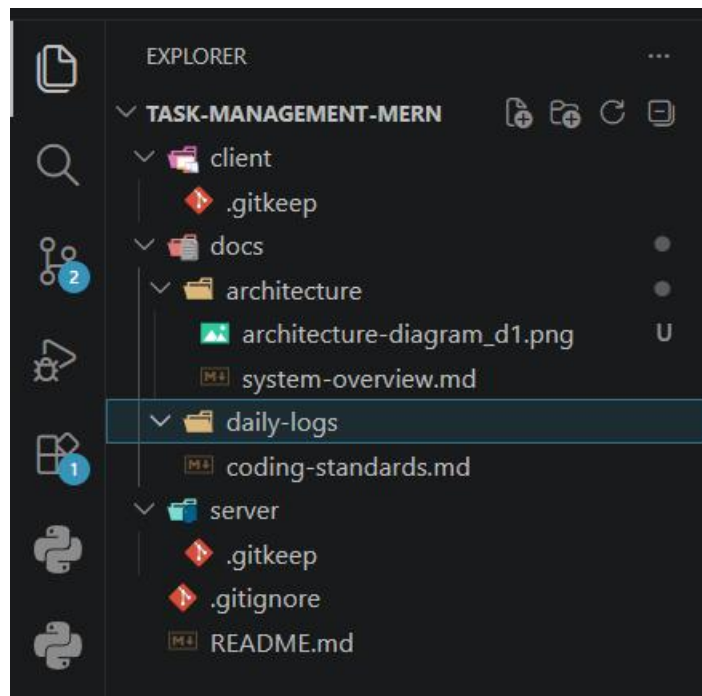


Figure 6.1 — Local workspace structure inside VS Code.

Both /client and /server are empty placeholders at this stage. Code setup begins from Day 2.

7. Challenges Faced

- SSH key configuration for GitHub — had to generate and register an SSH key before git push would work. The error was a permission denied (publickey) failure. Resolved by adding the key to GitHub account settings and verifying with `ssh -T git@github.com`.

```
shafi@Shafin-PC: ~/MyProject/task-management-mern (feature/project-setup)
$ git add .
$ git commit -m "docs: add architecture and coding standards"
$ git push --set-upstream origin feature/project-setup
[feature/project-setup eff06b7] docs: add architecture and coding standards
3 files changed, 79 insertions(+)
delete mode 100644 docs/architecture/.gitkeep
create mode 100644 docs/architecture/system-overview.md
create mode 100644 docs/coding-standards.md
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.09 KiB | 111.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/project-setup' on GitHub by visiting:
remote:   https://github.com/ShafinNigamana/task-management-mern/pull/new/feature/project-setup
remote:
To github.com:ShafinNigamana/task-management-mern.git
 * [new branch]   feature/project-setup -> feature/project-setup
branch 'feature/project-setup' set up to track 'origin/feature/project-setup'.

shafi@Shafin-PC: ~/MyProject/task-management-mern (feature/project-setup)
$ ssh -T git@github.com
Hi ShafinNigamana! You've successfully authenticated, but GitHub does not provide shell access.
```

Figure 7.1 — GitHub SSH authentication verification from local development environment.

No other major blockers on Day 1.

8. Next Steps (Day 2)

- Initialize the React client using Vite inside /client
- Set up the Express server with basic routing inside /server
- Connect to MongoDB Atlas and verify the connection
- Define the initial API structure and route files
- Set up ESLint and Prettier based on the coding standards doc

9. Conclusion

Day 1 was focused entirely on project setup and planning. The repository is initialized, the folder structure is in place, and the architecture and coding standards are documented. No feature code was written yet — that starts from Day 2. The foundation is set up in a way that should make it straightforward to start building the actual application.